Chapter 2

Instructions: Language of the Computer

Unsigned Binary Integers

- Given an n-bit number

$$x = x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- Range: 0 to $+2^n - 1$
- Example
  - $0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1011_2$
    $= 0 + \ldots + 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0$
    $= 0 + \ldots + 8 + 0 + 2 + 1 = 11_{10}$
- Using 32 bits
  - 0 to +4,294,967,295

Unsigned Binary Integers

- Given an n-bit number

$$x = x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- Range: 0 to $+2^n - 1$
- Example
  - $0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1011_2$
    $= 0 + \ldots + 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0$

Q1. A 12-bit unsigned integer is used. What is the range of values it can represent?

A1.

0~2^{12} – 1

0~4095

2s-Complement Signed Integers

- Given an n-bit number

$$x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- Range: -2^{n-1} to $+2^{n-1} - 1$
- Example
  - $1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1100_2$
    $= -1\times2^{31} + 1\times2^{30} + \ldots + 1\times2^2 + 0\times2^1 + 0\times2^0$
    $= -2{,}147{,}483{,}648 + 2{,}147{,}483{,}644 = -4_{10}$
- Using 32 bits
  - –2,147,483,648 to +2,147,483,647

2s-Complement Signed Integers

- Given an n-bit number

$$x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

8-bit Representation	+5	–5	Range
Unsigned	00000101	N/A	0~255
Sign-Magnitude	00000101	10000101	-127~127
One's Complement	00000101	11111010	-127~127
Two's Complement	00000101	11111011	-128~127

Using 32 bits

- –2,147,483,648 to +2,147,483,647

2s-Complement Signed Integers

- Bit 31 is sign bit
  - 1 for negative numbers
  - 0 for non-negative numbers
- $-(-2^{n-1})$ can't be represented
- Non-negative numbers have the same unsigned and 2s-complement representation
- Some specific numbers
  - 0: 0000 0000 … 0000
  - –1: 1111 1111 … 1111
  - Most-negative: 1000 0000 … 0000
  - Most-positive: 0111 1111 … 1111

2s-Complement Signed Integers

- Bit 31 is sign bit
  - 1 for negative numbers
  - 0 for non-negative numbers

Q2. Consider the following 8-bit binary number 11010110_2

a) What is its decimal value if it is interpreted as an unsigned integer?

b) What is its decimal value if it is interpreted as a two's-complement signed integer?

A2.

a) $128+64+16+4+2=214$

b) $-128+64+16+4+2=-42$

Signed Negation

- Complement and add 1
  - Complement means 1 → 0, 0 → 1

$$x + \bar{x} = 1111\ldots111_2 = -1$$

$$\bar{x} + 1 = -x$$

- Example: negate +2
  - +2 = 0000 0000 … 0010_2
  - –2 = 1111 1111 … 1101_2 + 1
    = 1111 1111 … 1110_2

Signed Negation

- Complement and add 1
  - Complement means 1 → 0, 0 → 1

$$x + \bar{x} = 1111\ldots111_2 = -1$$

$$\bar{x} + 1 = -x$$

Q3. Represent −18 as an 8-bit two's-complement number.

A3.

18=000100102

Invert: 11101101_2

Add 1: 11101110_2

Sign Extension

- Representing a number using more bits
  - Preserve the numeric value
- In MIPS instruction set
  - `addi`: extend immediate value
  - `lb`, `lh`: extend loaded byte/halfword
  - `beq`, `bne`: extend the displacement
- Replicate the sign bit to the left
  - c.f. unsigned values: extend with 0s
- Examples: 8-bit to 16-bit
  - +2: 0000 0010 => 0000 0000 0000 0010
  - –2: 1111 1110 => 1111 1111 1111 1110

Sign Extension

- Representing a number using more bits
  - Preserve the numeric value
- In MIPS instruction set
  - `addi`: extend immediate value
  - `lb`, `lh`: extend loaded byte/halfword
  - `beq`, `bne`: extend the displacement
- Replicate the sign bit to the left

Q4. Sign-extend the following 8-bit two's-complement number to 16 bits, and determine its decimal value.101101102

A4.

101101102 ⇒1111111101101102

−128+32+16+4+2=−74

Representing Instructions

- Instructions are encoded in binary
  - Called machine code
- MIPS instructions
  - Encoded as 32-bit instruction words
  - Small number of formats encoding operation code (opcode), register numbers, …
  - Regularity!
- Register numbers
  - $t0 – $t7 are reg's 8 – 15
  - $t8 – $t9 are reg's 24 – 25
  - $s0 – $s7 are reg's 16 – 23

Hexadecimal

- Base 16
  - Compact representation of bit strings
  - 4 bits per hex digit

0	0000	4	0100	8	1000	c	1100
1	0001	5	0101	9	1001	d	1101
2	0010	6	0110	a	1010	e	1110
3	0011	7	0111	b	1011	f	1111

- Example: eca8 6420
  - 1110 1100 1010 1000 0110 0100 0010 0000

Hexadecimal

- Base 16
  - Compact representation of bit strings
  - 4 bits per hex digit

0	0000	4	0100	8	1000	c	1100
1	0001	5	0101	9	1001	d	1101

Q5. Convert the following 32-bit binary number to hexadecimal:

$0010\ 1101\ 1010\ 0111\ 0100\ 1110\ 1001\ 0011_2$

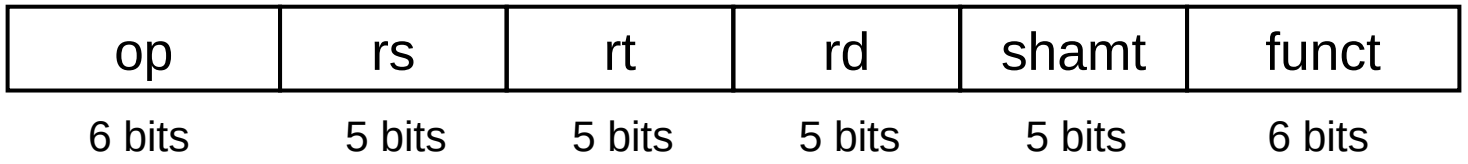
A5.

0010=2, 1101=D, 1010=A, 0111=7

0100=4, 1110=E, 1001=9, 0011=3

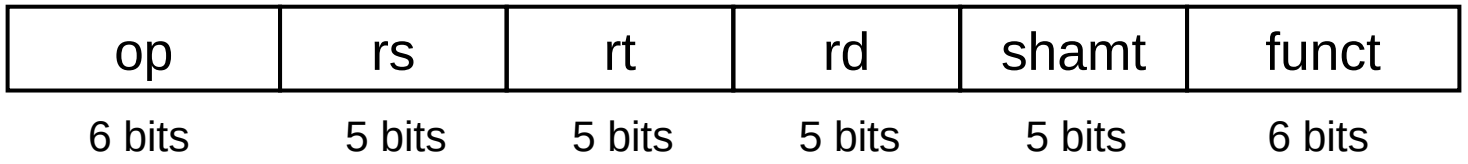
$2DA7\ 4E93_{16}$

MIPS R-format Instructions



- Instruction fields
  - op: operation code (opcode)
  - rs: first source register number
  - rt: second source register number
  - rd: destination register number
  - shamt: shift amount (00000 for now)
  - funct: function code (extends opcode)

MIPS R-format Instructions



- Instruction fields
  - op: operation code (opcode)
  - rs: first source register number
  - rt: second source register number

Q6. Consider the following MIPS instruction:
add $s2, $t0, $t1
Which registers correspond to rs, rt, and rd?
A6.
rs=$t0, rt=$t1, rd=$s2

R-format Example

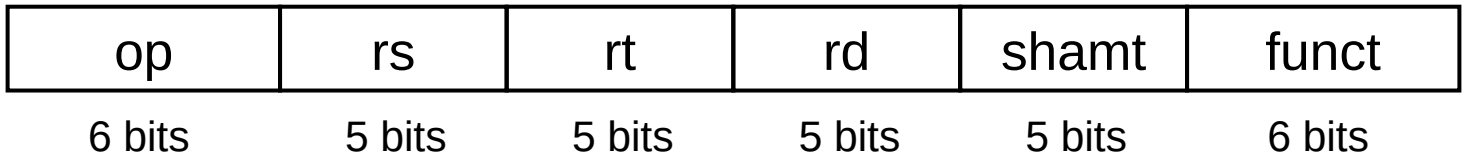
op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

```
add $t0, $s1, $s2
```

special	$s1	$s2	$t0	0	add
0	17	18	8	0	32
000000	10001	10010	01000	00000	100000

$$00000010001100100100000000100000_2 = 02324020_{16}$$

R-format Example



Q7. Given the following R-format fields, determine the corresponding MIPS instruction.

op	rs	rt	rd	shamt	funct
0	16	17	10	0	32

Given:

$s0=16, $s1=17, $t2=10

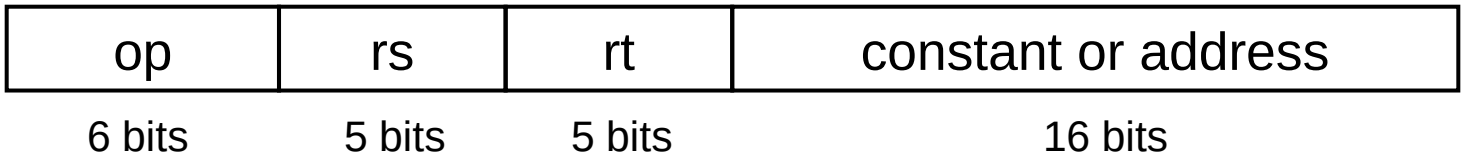
A7.

rs=$s0,rt=$s1,rd=$t2

funct = 32 => add

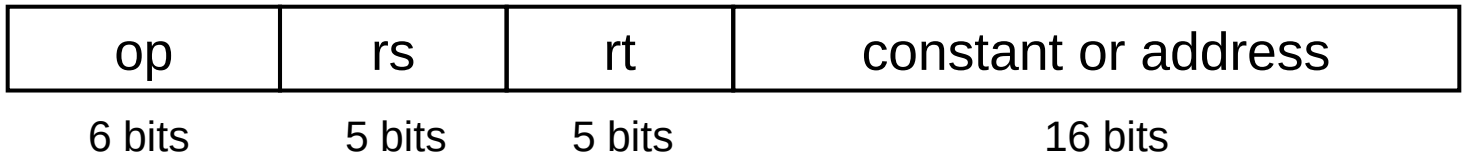
add $t2, $s0, $s1

MIPS I-format Instructions



- Immediate arithmetic and load/store instructions
  - rt: destination or source register number
  - Constant: -2^{15} to $+2^{15} - 1$
  - Address: offset added to base address in rs
- *Design Principle 4:* Good design demands good compromises
  - Different formats complicate decoding, but allow 32-bit instructions uniformly
  - Keep formats as similar as possible

MIPS I-format Instructions



- Immediate arithmetic and load/store instructions
  - rt: destination or source register number

Q8. A MIPS I-format instruction is 32 bits. If op, rs, and rt use 6, 5, and 5 bits, respectively:
How many bits remain for the constant/address?
What is the range of a signed constant?

A8.
32−(6+5+5)=16 bits
-2^{15} ~ 2^{15} - 1 =-32,768 ~ 32,767

Section 2-2 Quick Questions

Q1. For a 12-bit unsigned integer, determine the representable range.

Q2. Interpret 11010110_2 as both an unsigned and a two's-complement signed integer.

Q3. Represent −18 as an 8-bit two's-complement number.

Q4. Sign-extend 10110110_2 from 8 bits to 16 bits and determine its decimal value.

Section 2-2 Quick Questions

Q5. Convert 0010 1101 1010 0111 0100 1110 1001 0011_2 to hexadecimal.

Q6. For add $s2, $t0, $t1, identify the registers corresponding to rs, rt, and rd.

Q7. Given R-format fields op=0, rs=16, rt=17, rd=10, shamt=0, funct=32, determine the MIPS instruction.

Q8. For a 32-bit I-format instruction, determine the constant/address field width and its signed range.

Exercise 1 | Two’s-Complement Arithmetic

Assume 8-bit two’s-complement representation.

Calculate: $10010110_2 + 10110101_2$

(a) Convert both operands to signed decimal values.
(b) Perform the binary addition using 8 bits.
(c) Interpret the 8-bit result as a signed decimal integer.

Answer

(a) $10010110_2 = -106$; $10110101_2 = -75$
(b) $10010110_2 + 10110101_2 = 1\ 01001011_2$
(c) $01001011_2 = 75$

(a) Keeping the lower 8 bits gives 01001011_2.

Key Point: Two’s-complement addition uses ordinary binary addition, but signed overflow occurs when the true result is outside the representable range.

Exercise 2 | Representation & Sign Extension

Consider the 8-bit bit pattern 11011010_2.

(a) Interpret it as both an unsigned integer and a two's-complement signed integer.
(b) Extend it to 16 bits under both interpretations.
(c) Verify that each 16-bit result preserves the original numerical value.

Answer

(a) Unsigned: $11011010_2 = 218$
Signed: $11011010_2 = -38$
(b) Unsigned zero extension: $00000000\ 11011010_2$
(c) $00000000\ 11011010_2 = 218$

$11111111\ 11011010_2 = -38$
Therefore, both extensions preserve the original value.
Signed sign extension: $11111111\ 11011010_2$

Key Point: The same bit pattern can represent different values. Zero extension is used for unsigned values, while sign extension preserves a two's-complement signed value.

Exercise 3 | Decode a Machine Instruction

Given the 32-bit R-format machine instruction:

$000000\ 10010\ 10011\ 01000\ 00000\ 100000_2$
Assume $s2 = 18, $s3 = 19, $t0 = 8, and funct = 32 means add.
(a) Identify op, rs, rt, rd, shamt, and funct in decimal.
(b) Decode the complete MIPS assembly instruction.
(c) Convert the 32-bit machine instruction to hexadecimal.

Answer

(a) op = 0, rs = 18, rt = 19, rd = 8, shamt = 0, funct = 32
(b) rs = $s2, rt = $s3, rd = $t0
(c) $0000\ 0010\ 0101\ 0011\ 0100\ 0000\ 0010\ 0000_2$

$= 02534020_{16}$

(a) funct = 32 → add
(b) add $t0, $s2, $s3

Key Point: Decoding R-format machine code requires separating the six fields, interpreting register numbers and funct, then reconstructing the assembly instruction.

Exercise 4 | Encode an R-format Instruction

Encode the MIPS instruction: add $t2, $s0, $s1

$t2 = 10, $s0 = 16, $s1 = 17, op = 0, shamt = 0, and funct = 32.
(a) Determine the six R-format field values and write each field in binary.
(b) Construct the complete 32-bit machine instruction.
(c) Convert the 32-bit instruction to hexadecimal.

Answer

(a) op = 000000, rs = 10000, rt = 10001
rd = 01010, shamt = 00000, funct = 100000
(b) $000000\ 10000\ 10001\ 01010\ 00000\ 100000_2$
(c) $0000\ 0010\ 0001\ 0001\ 0101\ 0000\ 0010\ 0000_2$

$= 02115020_{16}$

Key Point: Encoding an R-format instruction proceeds from assembly operands to register numbers, fixed-width fields, a 32-bit bit pattern, and finally hexadecimal machine code.

Exercise 5 | Integrated R/I-format Analysis

Compare:

add $t0, $s1, $s2
addi $t0, $s1, −12
Assume $t0 = 8, $s1 = 17, $s2 = 18.
(a) Identify the instruction format and register roles for each instruction.
(b) Represent −12 as the 16-bit immediate field used by addi.
(c) Compare how the 32 bits are allocated in R-format and I-format, and explain why addi can encode a constant directly.

Answer

(a) add: R-format; rs = $s1, rt = $s2, rd = $t0
addi: I-format; rs = $s1, rt = $t0, immediate = −12

(b) +12 = 0000000000001100_2

(c) R-format: op(6) | rs(5) | rt(5) | rd(5) | shamt(5) | funct(6)

I-format: op(6) | rs(5) | rt(5) | constant/address(16)
I-format replaces rd, shamt, and funct with a 16-bit immediate field.
−12 = 1111111111110100_2

Key Point: R-format dedicates fields to register operations, while I-format uses a 16-bit field to encode an immediate constant or address offset directly in the instruction.